

FINAL CHALLENGE 2025 - JKU.WAD



Gianluca Galletti

2025/06/24

drl@ml.jku.at

LIT AI LAB, Institute for Machine Learning

Slides: Gianluca Galletti, Fabian Paischer

Schedule

- Leaderboard, results and common issues
- Student presentations
- Feedback
- Tournament

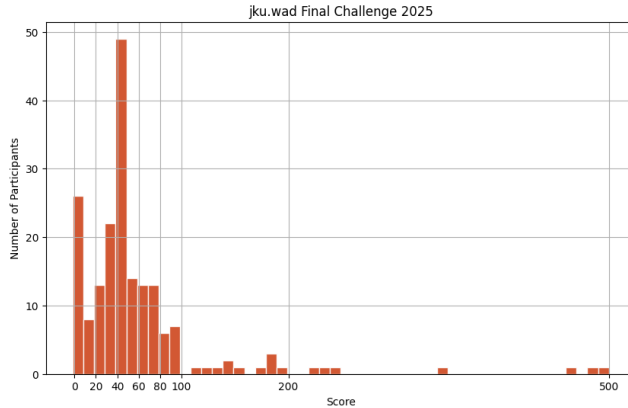
JKU.WAD **RESULTS**



Leaderboard

#	Name	DoomRLFinalChallenge2025	Total Submissions	Last Submission
1	Mini-Rainbow + QR	499.46	16	21 hours ago
2	Klem's Han Solo	479.8	6	23 hours ago
3	Dominatrix	460.9	18	20 hours ago
4	>:(	341.89	12	6 days ago
5	try4	247.26	4	27 days ago
6	Rip-And-Tear	234.61	1	20 days ago
7	no_movement_shooter	228.56	8	12 days ago
8	DoomQNetwork 1 Million	195.47	9	11 days ago
9	🐼🐼🐼	187.22	5	10 days ago
10	transfer	184.03	12	1 day ago

Grade distribution



Bugs / Server issues

- Many bugs in the initial phase → **sorry!**
- A few player config inconsistencies.
- Server problems in the final days: impossible to upload, 500 unauthorized, ...
 - ↪ we are investigating the reason.
 - ↪ cannot reproduce.

Sincere thanks to anyone who pointed out problems on moodle, github and email as they were encountered!

STUDENT PRESENTATIONS





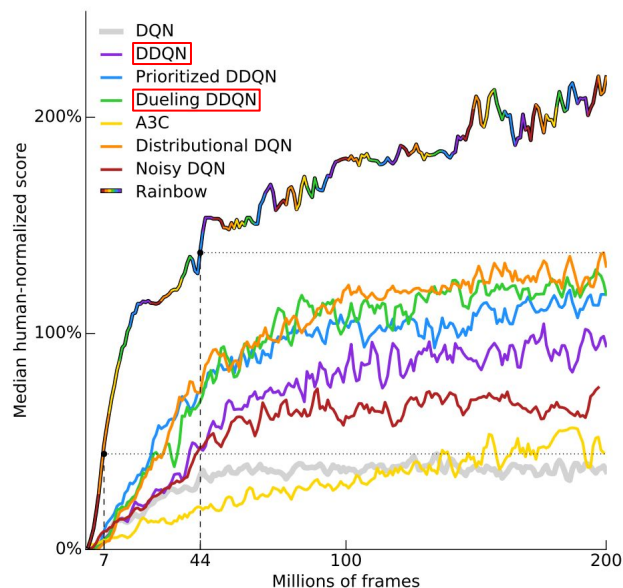
VizDoom Mini-Rainbow + QR

Dominik Baron

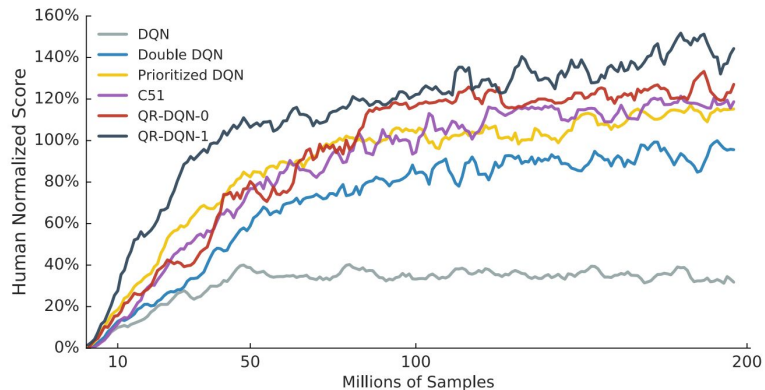


Rainbow: Combining Improvements in Deep Reinforcement Learning (2017)

Double DQN, Duelling Heads, N-Step Return



Distributional Reinforcement Learning with Quantile Regression (2017)



Architecture

- CoordConv
- Spatial Pyramid Pooling
- Duelling heads
- Quantile Regression (51 Quantiles)
- 1.75 million parameters total

```
def predict_quantiles(self, frame: torch.Tensor) -> torch.Tensor:
    """
    Returns (B, A, N_Q).
    """
    B = frame.size(0)
    coords = self.coords.unsqueeze(0).expand(B, 2, 128, 128)
    ego_lat = self.egocentric_encoder(torch.cat([frame[:, :5], coords], dim=1))
    map_lat = self.automap_encoder(torch.cat([frame[:, 5:], coords], dim=1))
    latent = self.shared_encoder(torch.cat([ego_lat, map_lat], dim=-1))

    v = self.v_head(latent).view(B, 1, N_QUANTS).expand(B, self.action_space, N_QUANTS)
    adv = self.adv_head(latent).view(B, self.action_space, N_QUANTS)
    return v + adv - adv.mean(dim=1, keepdim=True)

def forward(self, frame: torch.Tensor) -> torch.Tensor:
    """
    Returns Q-values = mean over quantiles, shape (B, A).
    """
    theta = self.predict_quantiles(frame)
    return theta.mean(dim=-1)
```

```
self.coords = nn.Parameter(torch.cat(
    [
        torch.linspace(-1, 1, 128).view(1, 128, 1).expand(1, 128, 128),
        torch.linspace(-1, 1, 128).view(1, 1, 128).expand(1, 128, 128)
    ],
    dim=0
), requires_grad=False)

self.egocentric_encoder = nn.Sequential(
    nn.Conv2d(5+2, 48, 8, stride=4), nn.ReLU(inplace=True),
    nn.Conv2d(48, 64, 4, stride=2), nn.ReLU(inplace=True),
    nn.Conv2d(64, 64, 3, stride=1), nn.ReLU(inplace=True),
    nn.Conv2d(64, 128, 3, stride=1, dilation=2), nn.ReLU(inplace=True),
    ResBlock(128),
    SPP((1, 2, 4)),
    nn.Flatten(),
)

self.automap_encoder = nn.Sequential(
    nn.Conv2d(3+2, 32, 8, stride=4), nn.ReLU(inplace=True),
    nn.Conv2d(32, 48, 4, stride=2), nn.ReLU(inplace=True),
    nn.Conv2d(48, 48, 3, stride=1), nn.ReLU(inplace=True),
    nn.Conv2d(48, 64, 3, stride=1, dilation=2), nn.ReLU(inplace=True),
    SPP((1, 2)),
    nn.Flatten(),
)

self.shared_encoder = nn.Sequential(
    nn.Linear(128 * 21 + 64 * 5, 256), nn.ReLU(inplace=True)
)

self.v_head = nn.Sequential(
    nn.Linear(256, 256), nn.ReLU(inplace=True),
    nn.Linear(256, N_QUANTS)
)

self.adv_head = nn.Sequential(
    nn.Linear(256, 512), nn.ReLU(inplace=True),
    nn.Linear(512, action_space * N_QUANTS)
)
```

Hyper-Parameters

- Reward scaled by 0.1 and clipped to $[0, 2]$ => ignores “hit taken” reward noise
- Gamma: 0.992
- Buffer size: 150k - 300k => saved as uint8 & without duplicates => 300k fits in 50gb
- Learning rate: $5e-5$ - $1e-7$, cosine annealing
- Batch size: 64, one batch every 4 environment step
- Epsilon-decay: ~ 0.996
- 3-step TD weight: 1.0; 1-step TD weight: 0.3
- Demonstrator (previous model) used to rollout trajectories at the beginning of training
- Around 10k episodes
- Epsilon & learning rate bump every $\sim 2k$ episodes.

How reliable are current LLMs for RL development? (o3 and Gemini 2.5 Pro Preview)

- Finding papers for improvement techniques
- Code review and finding certain bugs
- Generating traditional setups (does it well without major bugs but not always fully correct/efficient)
- Seeking advice when a traditional setup (implementation & hyperparameters) doesn't work
=> contradicting advice

Final Challenge Reinforcement Learning 2025: Playing Doom

Klemens Hauzenberger

The beginning: Score 85

- ▶ **PPO** (better stability and convergence properties than DQN)
-

- ▶ **Input:** $4 \times 128 \times 128$, RGB + depth map

- ▶ **Feature extractor:**

- ▶ Conv2D: 8×8 , stride 4 $\rightarrow$ LayerNorm $\rightarrow$ Leaky ReLU
- ▶ Conv2D: 4×4 , stride 2 $\rightarrow$ LayerNorm $\rightarrow$ Leaky ReLU
- ▶ Conv2D: 3×3 , stride 1 $\rightarrow$ LayerNorm $\rightarrow$ Leaky ReLU

- ▶ **Actor head:**

- ▶ Linear (9126×64) $\rightarrow$ Leaky ReLU $\rightarrow$ Linear (64×7)

- ▶ **Critic head:**

- ▶ Linear (9126×64) $\rightarrow$ Leaky ReLU $\rightarrow$ Linear (64×1)

- ▶ **Trainable parameters:** 1.3 million
-

- ▶ **No reshaping of rewards:** used the one of evaluation script

Observations from many failed attempts to improve score

- ▶ Doom Guy learns to hit but does not really learn to kill
 - ▶ No progress in training over many episodes
 - ▶ Reward for kill (+100) tends to destabilize training
 - ▶ Policy often becomes deterministic (mode collapse)
 - ▶ Reward reshaping toward more efficiency does not help
 - ▶ Penalty if not moving, i.e., stuck at walls or corners (-0.001)
 - ▶ Penalty for every shot that was not a hit (-0.002)
 - ▶ Regressive time penalty for steps between hits (-0.001)
 - ▶ Regressive time penalty for steps between kills (-0.002)
 - ▶ What else did not work (but should potentially work)
 - ▶ Adding labels as input for better detecting and aiming
 - ▶ Time-stacking frames to improve movement
-
- ▶ What was missing?
 - ▶ Parameter space probably (way) too large

The breakthrough: Score 480

- ▶ **Input:** $5 \times 128 \times 128$, RGB + labels + depth map
- ▶ **Feature extractor:**
 - ▶ Conv2D: 8×8 , stride 4 $\rightarrow$ LayerNorm $\rightarrow$ SiLU
 - ▶ Conv2D: 4×4 , stride 2 $\rightarrow$ LayerNorm $\rightarrow$ SiLU
 - ▶ Conv2D: 3×3 , stride 1 $\rightarrow$ LayerNorm $\rightarrow$ SiLU
- ▶ **Actor head:**
 - ▶ Linear (9126×32) $\rightarrow$ ReLU $\rightarrow$ Linear (32×7)
- ▶ **Critic head:**
 - ▶ Linear (9126×16) $\rightarrow$ ReLU $\rightarrow$ Linear (16×1)
- ▶ **Trainable parameters:** 0.6 million
- ▶ **Reward reshaping:** kill 25 and scheme from previous slide

Implementation details: PPO with GAE ($\gamma=0.99$, $\lambda=0.95$), normalized advantages, policy and value loss clipping (0.2), global gradient clipping (0.5), loss weighting: actor (1.0), critic (0.5), entropy (0.001); Adam (eps=1e-5), LR(features)=1e-4, LR(actor)=3e-4, LR(critic)=1e-4; cosine annealing over ~ 6000 steps (2e7 frames, 11h on Colab CPU); episode=2000 frames, epochs=4 over episode, batch size=250

Vizdoom Challenge

Peter Brandstetter - K51846528

Policy - Architecture

- SB3 CnnPolicy as base.
Overwritten / Extended using BaseFeaturesExtractor
- 1st Conv2d with big kernel
(kernel_size = 8)
- LeakyReLU activation
- BatchNorm2d between
Convolutions
- 20% Dropout on the Linear
Layer

```
class DoomCNN(BaseFeaturesExtractor):
    def __init__(self, observation_space: Box, features_dim: int = 512):
        super().__init__(observation_space, features_dim)
        n_input_channels = observation_space.shape[0]

        self.cnn = nn.Sequential(
            nn.Conv2d(n_input_channels, 32, kernel_size=8, stride=4, padding=0), # → 31x31
            nn.BatchNorm2d(32),
            nn.LeakyReLU(),
            nn.Conv2d(32, 64, kernel_size=4, stride=2, padding=0), # → 14x14
            nn.BatchNorm2d(64),
            nn.LeakyReLU(),
            nn.Conv2d(64, 64, kernel_size=3, stride=1, padding=0), # → 12x12
            nn.BatchNorm2d(64),
            nn.LeakyReLU(),
            nn.Flatten(),
        )

        with torch.no_grad():
            sample_input = torch.zeros(1, *observation_space.shape)
            n_flatten = self.cnn(sample_input).shape[1]

        self.linear = nn.Sequential(
            nn.Linear(n_flatten, features_dim),
            nn.LeakyReLU(),
            nn.Dropout(p=0.2)
        )

    def forward(self, observations: torch.Tensor) -> torch.Tensor:
        return self.linear(self.cnn(observations))
```

Environment

- Extended environment script for ROOM in Slade (added function to change bot health and bot speed during training) ... small changes in doom_env.py needed
- Vectorized env with 8 envs for training and 4 envs for evaluation ... each env with a different seed
- Chooosed seeds by playing VizDoom with pygame =) ... for many seeds it happened, that a bot immediately spawned on the agents spawn position (respawn takes around 100 steps and maybe punish agent for no reason)

Training and Evaluation

- Curriculum Learning with multiple stages (6 stages in total)
- Stages changed depending on the performance in the eval envs
- With every stage speed and health of the bots increased in both the train and eval envs
- Started with frame_skip of 4 for fast training, reduced down to 1 with stage change
- Cosine Annealing LR and Linear Decay for Clip Range
- Tensorboard for monitoring the performance <3

Rewards

- Extended the reward function in doom_env.py to return the list of rewards and not the sum ... logged every type of reward in Tensorboard ... Helped me to find out the reasons why the agent doesn't perform

```
# compute rewards
rwd_frag = 1.0 * delta_frag
rwd_damage = 0.01 * delta_damage_dealt
rwd_ammo_picked = 0.02 * max(0, delta_ammo_picked)
rwd_health_gained = 0.02 * max(0, delta_health_gained)
rwd_health_lost = -0.01 * max(0, delta_health_lost)
rwd_armor_gained = 0.01 * max(0, delta_armor)

if delta_moved > 0.001:
    rwd_moved = min(0.0001, 0.0001 * delta_moved) # scale the reward for moving
else:
    rwd_moved = -0.0025
```

FEEDBACK



TOURNAMENT



Tournament

- **Format:** PvP, 8 4 player deathmatch (FFA)
- **Map:** ~~TRNM~~ **ROOM.**
- **Rules:** single elimination bracket, with a single winner.
- **Score:** reweighted PvE score.
- **Agents:** onnx dumps from moodle.

HIGHLIGHTS



Thanks to everyone who
took part in the DLR Exercises!